Improving Adaptive Random Forests (ARFs) for Regression via Dynamic Weighting and Ensemble Resizing

Final Project Report

Authors: Filip Kucia, Bartosz Grabek 20 June 2025

Abstract

Research in data mining techniques often focuses on introducing a range of heuristics to improve the performance of algorithms used for non-stationary data streams. As traditional ensemble methods such as Adaptive Random Forests (ARFs) often suffer from limited flexibility, in this research we explore dynamic ensemble size management and its impact on model adaptation and performance. The experiments focus on adjusting the number of base learners based on performance metrics and drift signals, and assignments of prediction weights to trees with online error tracking. Furthermore, we investigate the potential of including an ensemble pruning strategy that removes the underperforming models tracked by proxy accuracy windows. The solution is benchmarked on both real-world and synthetic data streams with different types of concept drift. Preliminary results indicate that in certain conditions such modifications can significantly improve the predictive performance of ensemble models for regression tasks in the context of data streams.

Contents

1 Introduction

1.1 Background and Motivation

The ever-increasing supply and demand for real-time data processing has accelerated the need for algorithms based on *online learning*, a machine learning (ML) paradigm where the model learns in an incremental fashion, updated continuously on new arriving instances. Unlike traditional ML methods based on batch learning (and datasets), stream mining requires models to operate under strict time and memory constraints given the volume and velocity of Big Data. Additionally, these algorithms have to tackle a phenomenon known as *concept drift*, during which the underlying data distribution evolves over time, potentially degrading model performance if not properly addressed.

One of the foundational algorithms for data stream learning is the **Hoeffding Tree (HT) / Very Fast Decision Tree (VFDT)** [1], which uses the Hoeffding bound to incrementally grow decision tress with high split quality guaranteed by the underlying statistical properties. For regression tasks, the **Fast Incremental Model Tree with Drift Detection (FIMT-DD)** [2] algorithm was introduced as an extension of VFDTs that uses linear models at the leaves and integrates explicit drift detection mechanisms (Page-Hinkley Test) to better handle concept drifts. However, the aforementioned single-model solutions often lack the ability to handle abrupt or recurring changes in data distribution. As a consequence, **ensemble methods** have become a commonplace standard in stream learning due to their ability to combine independent models (a.k.a. *base learners*) to make predictions collectively, for instance using majority vote. Such aggregations have proved to hold several key advantages over single-model methods:

- **They increase the robustness of the system**, the poor performance of a single model in an ensemble can be compensated by improved performance of other models, which reduces the risk of failure from the combined approach
- **They improve accuracy** as the aggregation of predictions via majority vote or weighted average often outperforms single models, not only in dynamically changing environments
- **They hold natural adaptation abilities**, incorporating new mechanisms for training on recent data, and discarding the outdated learners, which helps with handling concept drift
- **They enhance the model diversity** allowing the overall model to capture numerous patterns at once and better adapt to various types of concept drift

One of the popular examples of an ensemble approach in stream mining is **Adaptive Random Forest** (ARF[1]), originally designed for classification tasks [3]. The algorithm was later extended to ARF-Reg[2] to tackle regression tasks as well [4]. Both ARF and ARF-Reg ensemble models, to which we refer to as ARF family of models, train multiple Hoeffding Trees on reweighted samples (usually via Poisson sampling with $\lambda = 6$) and use **ADWIN drift detectors** [5] in each of the base learners. When a particular tree detects a drift, it is replaced by a new one trained on most recent data, which allows the ensemble to adapt selectively over time. The family of ARF models have

[1] we will refer to the original ARF algorithm for classification as ARF

[2] we will refer to the ARF algorithm for regression as ARF-Reg, following the notation in the original paper

been widely adopted in stream learning as they demonstrate strong predictive performance, and it is easy to integrate different drift detection techniques within them. Despite their effectiveness, ARF family of models, similarly to other ensemble methods, suffer from some structural limitations. One of such limitations is the fixed ensemble size (number of base learners), which, regardless of input stream complexity, may lead to overfitting or non-efficient use of computing resources. Furthermore, in ARF-Reg all trees contribute equally to the prediction (simple averaging) - such approach may be suboptimal as it ignores the fact that some learners may be more accurate or better adapted to the current data distribution [3]. Interestingly, this is contrary to ARFs for classification where trees contribute to the final prediction through weighted majority voting, that is each tree's weight is determined by its test-then-train accuracy: $\frac{c_l}{n_l}$, c_l and n_l denoting the number of correct and total predictions since the tree's last reset [3]. Lastly, the family of ARF models lacks advanced pruning mechanisms, which may degrade their performance by allowing outdated or redundant trees to persist.

To overcome some of the limitations of the original ensemble models, recent research in stream mining focuses on incorporating special heuristics, extensions to the state-of-the-art algorithms to increase their performance and robustness to concept drift. One such example is **Robust Online Self-Adjusting Ensemble (ROSE)** [6], proposed for imbalanced classification in 2022. ROSE dynamically tunes the composition of the ensemble by evaluating base learner performance, trains background models on recent data, and performs batch-wise ensemble replacement to adapt quickly to drift. Furthermore, it incorporates additional mechanisms such as *dynamic feature subspaces*, meaning each base learner is trained on a randomly sized subset of features, and *per-class sliding windows* for improved specialization and responsiveness to concept drift and class imbalance.

Inspired by these ideas, in this work, we investigate an enhanced ARF-Reg ensemble models for tackling regression that test several mechanisms, including dynamic resizing of the ensemble, calibration of base learners' weights based on performance tracking, and pruning techniques for removing underperforming learners with proxy error windows. Our goal is to investigate these features and utilize them to develop a more flexible and efficient alternative to standard ARF-Reg with better adaptiveness to concept drift.

1.2 Project Scope & Research Goals

The project focuses on improving the robustness and adaptability of ensemble-based learning algorithms, namely ARF-Regs, for regression tasks on evolving data streams. The core objective is to investigate and incorporate heuristics that addresses the limitations of current state-of-the-art ARF-Regs which are fixed (static) ensemble size, equal voting scheme and lack of efficient dynamic pruning. The scope of the project includes:

- Developing extended versions of the Adaptive Random Forest for regression (extended ARF-Regs) to serve as a basis for further enhancements.

[3] `ARFRegressor` in `river` library, in contrast with the original algorithm, uses `HoeffdingTreeRegressor` instead of `FIMT-DD` and it does allow to use weighted voting by using a user-specified metric, e.g. mean squared error (MSE). To retain the original behaviour use `disable_weighted_vote=True`. Reference: `https://riverml.xyz/0.18.0/api/forest/ARFRegressor/`

- investigating and applying drift detection mechanism to maintain model relevance in a non-stationary environment
- designing and evaluating strategies for dynamic ensemble resizing, prediction weighting based on online error tracking and custom pruning techniques
- benchmarking performance of the proposed solutions across real-world and synthetic data streams

The deliverables of the project include this report, `GitHub` repository with Python code implementing and testing the above project goals (see Appendix A), and a PDF presentation giving an overview of the project results.

2 Methodology & Experimental Setup

2.1 Standard ARF-Reg

As a baseline for our experiments we implement an extended version of Adaptive Random Forest regressor (`ARFRegressor`), to which we refer as Standard ARF. It retains the three key aspects of the original ARF-Reg [4] such as:

1. Inducing diversity through resampling
2. Inducing diversity through randomly selecting subsets of features for node splits
3. Drift detectors per base tree (base learner), which cause selective resets in response to drifts

In comparison with the original algorithm proposal, our solution differs slightly in implementation:

- the `HoeffdingTreeRegressor` is used as a base learner, instead of `FIMT-DD`
- the deviation of the predictions to the target is monitored and normalized (to $[0, 1]$ range) to meet ADWIN drift detector's requirements

This setup simplifies integration with existing `river` library capabilities, and is in place to ensure compatibility with general-purpose regression trees that are widely supported and efficient in practice. While `FIMT-DD` offers stronger theoretical guarantees for linear modeling, `HoeffdingTreeRegressor` oftentimes has better performance.

To better illustrate how each individual tree in the ensemble is trained, RFTreeTrain algorithm (see Algorithm 1), which describes the core update routine used in Standard ARF. Each base learner is an instance of a regression tree trained incrementally using the logic of a Hoeffding Tree. For clarity, we refer to such a tree generically as `RFTree` — a shorthand for a regression-capable stream tree. In our implementation, this corresponds directly to `HoeffdingTreeRegressor` from the `river` library.

An `RFTree` is trained incrementally on a single data instance (x, y), where t denotes the regression tree, and m is the number of features randomly selected (from the total feature set M) to be considered when evaluating potential split candidates at each node. The process starts by drawing a sample weight k from a Poisson distribution with $\lambda = 6$ - this emulates bagging and controls

how many times a specific instance `(x, y)` is used to train the tree. During the training step (if $k > 0$) the function finds the appropriate leaf node in the tree and updates statistics at that leaf. If enough instances have been seen, controlled by Grace Period (GP), the tree checks if a split should be attempted. If a split was made successfully, the leaf is replaced by new child nodes.

Algorithm 1 RFTree Train

```
function RFTreeTrain(m, t, x, y)
    k ← Poisson(λ = 6)
    if k > 0 then
        l ← FindLeaf(t, x)
        UpdateLeafCounts(l, x, k)
        if InstancesSeen(l) ≥ GP then
            AttemptSplit(l)
            if DidSplit(l) then
                CreateChildren(l, m)
            end if
        end if
    end if
end function
```

Algorithm 2 ARF-Reg

```
function ARF_Reg(m, n, δ_w, δ_d)
    T ← CreateTrees(n)                          ▷ Initialize n trees
    W ← InitWeights(n)                          ▷ Performance weights
    B ← ∅                                       ▷ Background models
    while HasNext(S) do
        (x, y) ← next(S)
        for all t ∈ T do
            ŷ ← predict(t, x)
            W(t) ← P(W(t), ŷ, y)
            RFTreeTrain(m, t, x, y)
            if c(δ_w, t, x, y) then             ▷ Warning detected
                b ← CreateTree()
                B(t) ← b
            end if
            if c(δ_d, t, x, y) then             ▷ Drift confirmed
                t ← B(t)                        ▷ Replace with background model
            end if
        end for
        for all b ∈ B do                        ▷ Update background models
            RFTreeTrain(m, b, x, y)
        end for
    end while
end function
```

The ARF-Reg algorithm [4] (see Algorithm 2) is an ensemble consisting of n base learners, each an RF tree. Each tree t is associated with a performance score $W(t)$, which is updated over time based on its predictive accuracy. For each incoming instance from stream S, every tree in the ensemble first performs a prediction and its performance score is updated using a function

like exponentially smoother error. The instance is used to incrementally train the tree through a subroutine `RFTreeTrain` described earlier. To enable adaptability to concept drift, each tree is equipped with a drift detection mechanism such as ADWIN. When a warning of a drift is raised, a new background tree b is created and begins learning from the same data stream in parallel to the main tree. This background model serves as a candidate replacement. If the detector confirms a concept drift (based on threshold δ_d), the main tree t is immediately replaced by its corresponding background model $B(t)$.

2.2 Dynamically Weighted Adaptive Random Forest for Regression (DWARF)

We introduce an extension to the standard ARF-Reg called **Dynamically Weighted Adaptive Random Forest for regression (DWARF)** (see Algorithm 3). This version of ARF for regression introduces a custom mechanism for dynamic prediction weighting that adjusts the contribution of each tree based on its recent performance. In addition to maintaining base metrics $W(t)$ for drift detection, each tree t is assigned a dynamic performance score $S_p(t)$ which is updated online using a function U_{dyn}. The score reflects how well the tree has been predicting recently and is used to weigh its contribution during ensemble prediction. Unlike standard ARF-Reg where all base models contribute equally, this approach allows the ensemble to emphasize better-performing trees, making it more robust under concept drift.

The function U_{dyn} operates by first calculating the absolute error $e_{abs} = |y - \hat{y}|$ and comparing it to a dynamic threshold T. This threshold is defined based on either the tree's running Mean Absolute Error (MAE_t) or its running error standard deviation (σ_t). The term "running" signifies that these are online statistics, updated incrementally with each new instance without needing to store the entire error history. This makes the approach memory-efficient and suitable for data streams. The subscript $_t$ is crucial, as it indicates that each tree in the ensemble maintains its own independent performance statistics, allowing for a personalized evaluation.

The choice between these two statistics provides different strategies for penalizing trees.

- **MAE-based Threshold ($T = \text{MAE}_t \cdot (1 + \epsilon_{th})$):** This approach penalizes trees whose current error is significantly worse than their own historical **average performance**. It is effective at identifying trees that have suddenly started performing poorly, perhaps because a concept drift has invalidated the patterns they learned.
- **STD-based Threshold ($T = \sigma_t \cdot \epsilon_{th}$):** This approach penalizes trees that produce an error of unusually large magnitude relative to their typical error **consistency**. It is less about the average error and more about stability and reliability.

For the experiments, the STD-based threshold ('error_mode="std"') is selected. as default This choice prioritizes penalizing trees that produce unstable or highly variable predictions, aiming to improve the overall robustness of the ensemble. If an error is below this personalized threshold ($e_{abs} \leq T$), the tree's score is improved via multiplication ($S_p(t) \leftarrow S_p(t) \cdot 0.9$); otherwise, it is penalized ($S_p(t) \leftarrow S_p(t) \cdot 1.1$). A lower score signifies better performance, leading to a higher weight in the final ensemble prediction.

Algorithm 3 DWARF. Ensemble prediction uses weights derived from $S_p(t)$. **Symbols:** m: set of features; n: #trees; δ_w: warn. thr.; δ_d: drift thr.; $c(\cdot)$: change detect.; S: data stream; B: bkg. trees; $W(t)$: Tree t base metric (for drift); $P(\cdot)$: Base metric estim.; $S_p(t)$**: Tree** t **dynamic perf. score (for prediction weighting)**; $U_{dyn}(\cdot)$**: Dynamic score update fn.**; ϵ_{th}**: error threshold factor for dyn. weighting**.

function DWARF($m, n, \delta_w, \delta_d, \epsilon_{\mathbf{th}}$)
  $T \leftarrow$ CreateTrees(n)
  $W \leftarrow$ InitWeights(n) ▷ Init base metrics for drift detection
  $S_p \leftarrow$ **InitDynamicScores**(n) ▷ **Init dynamic perf. scores**
  $B \leftarrow \emptyset$
  while HasNext(S) **do**
    $(x, y) \leftarrow$ next(S)
    for all $t \in T$ **do**
      $\hat{y} \leftarrow$ predict(t, x)
      $W(t) \leftarrow P(W(t), \hat{y}, y)$ ▷ Update base metric for $c(\cdot)$
      $S_p(t) \leftarrow U_{dyn}(S_p(t), \hat{y}, y, \epsilon_{th})$ ▷ **Update dynamic score**
      RFTreeTrain(m, t, x, y) ▷ Train t
      if $c(\delta_w, t, x, y)$ **then** ▷ Warn detect?
        $b \leftarrow$ CreateTree() ▷ New bkg tree b
        $S_p(b) \leftarrow$ **InitDynamicScore**() ▷ **Init score for bkg tree**
        $B(t) \leftarrow b$
      end if
      if $c(\delta_d, t, x, y)$ **then** ▷ Drift detect?
        $t \leftarrow B(t)$ ▷ Replace $t \leftarrow B(t)$
        ResetBaseMetric($W(t)$) ▷ **Also reset** $W(t)$ **for new tree**
        ResetDynamicScore($S_p(t)$) ▷ **Reset dynamic score**
      end if
    end for
    for all $b \in B$ **do** ▷ Train bkg trees
      RFTreeTrain(m, b, x, y)
      $S_p(b) \leftarrow U_{dyn}(S_p(b), \mathbf{predict}(b, x), y, \epsilon_{th})$ ▷ **Update bkg tree score**
    end for
  end while
end function

2.3 ARF with Ensemble Resizing (ARFER)

Algorithm 4 ARFER - ARF with Ensemble Resizing. B: bkg. trees; $W(t)$: Tree t base metric (for pruning/weighting); $P(\cdot)$: Base metric estim.; N_{max}**: max #trees**; τ_{acc}**: acc. drop thr. for pruning**; ϵ_{prune}**: error thr. for proxy acc.**; M_w**: monitor window**; $A_w(t)$**: proxy acc. window for** t; $A_{warn}(t)$**: acc. at warning for** t; S_{warn}**: set of warned trees**.

function ARFER($m, n_{init}, \delta_w, \delta_d, \mathbf{N_{max}}, \tau_{\mathbf{acc}}, \epsilon_{\mathbf{prune}}, \mathbf{M_w}$)
  $T \leftarrow$ CreateTrees(n_{init})
  $W \leftarrow$ InitBaseMetrics($|T|$) ▷ For pruning/weighting
  $A_w, A_{warn}, S_{warn} \leftarrow$ **InitPruningState**($|T|, M_w$)
  $B \leftarrow \emptyset$
  while HasNext(S) **do**
    $(x, y) \leftarrow$ next(S)
    for all $t \in T$ **do** ▷ Iterate over a copy or manage indices carefully if T changes
      $\hat{y} \leftarrow$ predict(t, x)
      $W(t) \leftarrow P(W(t), \hat{y}, y)$ ▷ Update base metric
      UpdateProxyAccuracyWindow($A_w(t), |\hat{y} - y|, \epsilon_{prune}$)
      RFTreeTrain(m, t, x, y)
      if $c(\delta_w, t, x, y)$ **then** ▷ Warn detect?
        RecordWarningState($t, S_{warn}, A_w(t), A_{warn}(t)$)
        $b \leftarrow$ CreateTree(); $B(t) \leftarrow b$ ▷ Init bkg tree
        InitPruningStateForNewTree(b, A_w, A_{warn})
      end if
      if $c(\delta_d, t, x, y)$ **then** ▷ Drift detect on t?
        attempted_add $\leftarrow$ **false**
        if $B(t)$ exists and $|T| < N_{max}$ **then**
          AddTreeToEnsemble($T, B(t), W, A_w, ...$)
          attempted_add $\leftarrow$ **true**; $B(t) \leftarrow$ null
        else if $B(t)$ exists and $|T| \geq N_{max}$ **then**
          $t_{worst} \leftarrow$ FindWorstTree(T, W)
          if t_{worst} exists **then**
            PruneTree($T, t_{worst}, W, A_w, S_{warn}, ...$)
            AddTreeToEnsemble($T, B(t), W, A_w, ...$)
            attempted_add $\leftarrow$ **true**; $B(t) \leftarrow$ null
          end if
        end if
        if not attempted_add or $B(t)$ **did not exist/not added then**
          $T \leftarrow$ ReplaceTreeInPlace(t, NewTree(), $W, A_w, S_{warn}, ...$)
        end if
        ResetDetector($c(\delta_d, \cdot)$ **for original** t)
      end if
    end for
    PruneExcessTrees($T, N_{max}, W, A_w, S_{warn}, ...$) ▷ If $|T| > N_{max}$
    PruneOnAccuracyDrop($T, S_{warn}, A_w, A_{warn}, \tau_{acc}, W, ...$)
    for all $b \in B$ **do** ▷ Train existing bkg trees
      RFTreeTrain(m, b, x, y)
      UpdateProxyAccuracyWindow($A_w(b), |$**predict**$(b, x) - y|, \epsilon_{prune}$)
    end for
  end while
end function

We propose yet another extension to Standard ARF (ARF-Reg) called **ARF with Ensemble Resizing (ARFER)** (see Algorithm 4). This version of ARF introduces the mechanism of dynamic ensemble resizing, which allows the number of learners to grow and shrink between a defined minimum and a maximum size (N_{max}) based on performance and concept drift. To facilitate these dynamic decisions, ARFER enhances the monitoring of each tree. In addition to a base performance metric $W(t)$ (e.g., MAE), each tree maintains a "proxy accuracy" window, $A_w(t)$. This window of size M_w tracks the proportion of recent predictions whose absolute error falls below a defined threshold, ϵ_{prune}, serving as a lightweight measure of recent reliability.

This enhanced monitoring enables a sophisticated, two-stage response to concept drift. First, when a lenient 'warning_detector' signals a potential change (δ_w), the algorithm proactively creates a background learner $B(t)$ and, crucially, records the tree's current proxy accuracy as its baseline performance at the time of warning, $A_{warn}(t)$. Second, upon a confirmed change from a more sensitive 'drift_detector' (δ_d), the algorithm attempts to promote the background learner. If the ensemble is not at its maximum size ($|T| < N_{max}$), the background tree is added. If the ensemble is full, the worst-performing tree (based on $W(t)$) is pruned to make space for it. However, a background learner may not always be available, as the warning and drift detectors operate as independent, parallel processes. An abrupt drift can trigger the sensitive 'drift_detector' before the more conservative 'warning_detector' has initiated a background tree. In such cases where a background learner cannot be promoted, the now-outdated tree is simply reset to its initial state.

Furthermore, ARFER employs two proactive pruning heuristics. The primary method leverages the stored warning-time accuracy: if a warned tree's current proxy accuracy, $A_w(t)$, drops significantly below its recorded baseline $A_{warn}(t)$, the tree is deemed to have a sustained performance degradation and is pruned, provided the ensemble remains above its minimum size. Secondly, as a safeguard, if the ensemble size ever exceeds N_{max} after adding models, the worst-performing tree is immediately removed. Through these mechanisms of reactive adaptation and proactive pruning, ARFER continuously allocates capacity to models that better reflect the current data distribution, enhancing adaptability and robustness in non-stationary settings.

Additionally, pruning is performed in two ways:

1. if the ensemble exceeds N_{max} the worst-performing tree is removed
2. if a tree's accuracy drops significantly relative to its recorded accuracy at the last warning

Through these mechanisms, ARFER with dynamic ensemble resizing continuously allocates capacity to models that better reflect the current data distribution, enhancing adaptability and robustness in non-stationary settings.

2.4 Dynamically Weighted Adaptive Random Forest with Ensemble Resizing (DWARFER)

This final new variant of ARF, which we call **DWARFER**, integrates both dynamic prediction weighting (from DWARF) and dynamic ensemble resizing (from ARFER) to possibly improve adaptability and accuracy in evolving data streams (see Algorithm 5).

Algorithm 5 DWARFER. Prediction uses $S_p(t)$ derived weights. **Symbols:** m: feat./split; n_{init}: init. #trees; δ_w, δ_d: thr.; $c(\cdot)$: change detect.; $W(t)$: base metric; $P(\cdot)$: metric estim.; $S_p(t)$**: dyn. perf. score (0.9/1.1 rule)**; $U_{dw}(\cdot)$**: dyn. weight score update fn.**; ϕ_{dw}**: dyn. weight err. factor**; N_{max}**: max #trees**; τ_{acc}**: acc. drop thr.**; ϵ_{prune}**: proxy acc. err. thr.**; M_w**: monitor win.**; $A_w(t)$**: proxy acc. win.**; $A_{warn}(t)$**: acc. at warn.**; S_{warn}**: warned set**.

function DWARFER($m, n_{init}, \delta_w, \delta_d, \phi_{\mathbf{dw}}, \mathbf{N_{max}}, \tau_{\mathbf{acc}}, \epsilon_{\mathbf{prune}}, \mathbf{M_w}$)
  $T \leftarrow$ CreateTrees(n_{init})
  $W \leftarrow$ InitBaseMetrics($|T|$)
  $S_p \leftarrow$ **InitDynPerfScores**($|T|$) ▷ **For 0.9/1.1 weighting**
  $A_w, A_{warn}, S_{warn} \leftarrow$ **InitPruningState**($|T|, M_w$)
  $B \leftarrow \emptyset$
  while HasNext(S) **do**
    $(x, y) \leftarrow$ next(S)
    for all $t \in T$ **do** ▷ Manage indices if T changes
      $\hat{y} \leftarrow$ predict(t, x)
      $W(t) \leftarrow P(W(t), \hat{y}, y)$ ▷ Update base metric (for worst tree)
      $S_p(t) \leftarrow U_{dw}(S_p(t), \hat{y}, y,$ **ErrorStats**$(t), \phi_{dw})$ ▷ **Update 0.9/1.1 score**
      UpdateProxyAccuracyWindow$(A_w(t), |\hat{y} - y|, \epsilon_{prune})$
      RFTreeTrain(m, t, x, y)
      if $c(\delta_w, t, x, y)$ **then** ▷ Warn detect?
        RecordWarningState$(t, S_{warn}, A_w(t), A_{warn}(t))$
        $b \leftarrow$ CreateTree(); $B(t) \leftarrow b$
        InitDynPerfScoreForNewTree$(S_p(b))$; **InitPruningStateForNewTree**(b, A_w, A_{warn})
      end if
      if $c(\delta_d, t, x, y)$ **then** ▷ Drift detect on t?
        attempted_add $\leftarrow$ **false**
        if $B(t)$ exists and $|T| < N_{max}$ **then**
          AddTree$(T, B(t), W, S_p, A_w, ...)$ ▷ **Also inits S_p for new tree**
          attempted_add $\leftarrow$ **true**; $B(t) \leftarrow$ null
        else if $B(t)$ exists and $|T| \geq N_{max}$ **then**
          $t_{worst} \leftarrow$ FindWorstTree(T, W)
          if t_{worst} exists **then**
            PruneTree$(T, t_{worst}, W, S_p, A_w, S_{warn}, ...)$
            AddTree$(T, B(t), W, S_p, A_w, ...)$
            attempted_add $\leftarrow$ **true**; $B(t) \leftarrow$ null
          end if
        end if
        if not attempted_add or $B(t)$ **did not exist/not added then**
          $T \leftarrow$ **ReplaceTree**$(t,$ **NewTree**$(), W, S_p, A_w, S_{warn}, ...)$ ▷ **Resets** $S_p(t)$
        end if
        ResetDetector$(c(\delta_d, \cdot)$ **for original** $t)$
      end if
    end for
    PruneExcessTrees$(T, N_{max}, W, S_p, A_w, S_{warn}, ...)$
    PruneOnAccuracyDrop$(T, S_{warn}, A_w, A_{warn}, \tau_{acc}, W, S_p, ...)$
    for all $b \in B$ **do** ▷ Train existing bkg trees
      RFTreeTrain(m, b, x, y)
      $\hat{y}_b \leftarrow$ predict(b, x)
      $S_p(b) \leftarrow U_{dw}(S_p(b), \hat{y}_b, y,$ **ErrorStats**$(b), \phi_{dw})$
      UpdateProxyAccuracyWindow$(A_w(b), |\hat{y}_b - y|, \epsilon_{prune})$
    end for
    UpdateAllDynamicPredictionWeights$(S_p,$ **PredictionWeights**$)$
  end while
end function

2.5 Selection of Data Streams

Synthetic Data

To evaluate the performance of both the Standard ARF and its proposed variants, we selected three synthetic regression data streams from the `river.datasets.synth` module.

Name	Source	Drift	Description
`friedman`	`Friedman(seed=1)`	No	Nonlinear regression benchmark with 10 numeric features. The target is a nonlinear combination of the most relevant features with added noise. Serves as a static baseline.
`friedman_drift`	`FriedmanDrift(seed=1)`	Yes	Gradual concept drift introduced by modifying the relevance of features over time. Mimics slowly evolving data distributions.
`planes2d`	`Planes2D(seed=1)`	Yes	A piecewise-defined regression task over a 2D input space. The target function exhibits abrupt changes between planar regions, making it suitable for evaluating drift detection and ensemble adaptability.

Table 1: Summary of selected synthetic data streams used in the evaluation.

The selected streams provide a controlled but diverse testing ground:

- **Friedman** offers a static, moderately complex baseline where ensemble models should remain stable.
- **FriedmanDrift** introduces gradual drift, allowing us to assess how well models adapt to slowly changing relationships.
- **Planes2D** features abrupt changes in the regression surface, simulating sudden shifts common in dynamic environments (e.g., fault detection, changing user behavior).

Real-World Data

This section introduces the real-world datasets relevant to this work. The choice of the datasets was inspired by the previous research on evaluation of regression on evolving data streams [7] .The **Abalone** dataset [8], from the UCI Machine Learning Repository, contains physical attributes of abalones for predicting their age. The **Bike Sharing** dataset [9] is another commonly used regression dataset, providing data on bike rental demand with environmental and seasonal features. Derived from the 1990 U.S. Census, the **House8L** dataset [10] is used for predicting median house prices. The **Superconductivity** dataset [11] includes data for predicting the critical temperature of superconductors based on material properties. Table 2 provides a summary of these datasets.

2.6 Evaluation

The validation of the proposed DWARF, ARFER, and DWARFER algorithms followed the **prequential (or test-then-train) evaluation methodology**. To isolate the impact of each pro-

Table 2: Real-World Dataset Overview

DATASET	# INSTANCE	# FEATURE
Abalone	4,177	8
Bike Sharing	17,379	12
House8L	22,784	8
Superconductivity	21,263	82

posed enhancement, a comparative evaluation was conducted against the standard extended ARF-Reg (`ARFRegressor`) as a baseline. The experimental conditions were strictly controlled to ensure a fair and reproducible comparison. All algorithms were initialized with an ensemble of **10 base learners** and executed in a **single, deterministic run** per dataset, with the random **seed** for all models fixed to **42**. To guarantee that each model was exposed to an identical sequence of data, every stream was duplicated using the `itertools.tee` function.

The following regression metrics were used to monitor performance:

- **Mean Absolute Error (MAE)** – measures average absolute deviation from true values,
- **Root Mean Squared Error (RMSE)** – penalizes larger errors more heavily than MAE,
- **Coefficient of Determination (R^2)** – evaluates the proportion of variance explained by the model

These metrics were updated incrementally using River's streaming-compatible metric interfaces. In addition to reporting final scores, we plotted the MAE over time to illustrate how models adapt to incoming data. For the ARFER, the number of active trees was also tracked and visualized to highlight the behavior of its dynamic ensemble resizing mechanism.

All results were stored in a CSV file and are accompanied by ARFF exports of the evaluated streams to support reproducibility.

3 Results & Discussion

Table 3: Mean Absolute Error (MAE) Results. Lower is better. Bold indicates the best performance per dataset.

Data stream	ARFStandard	DWARF	ARFER	DWARFER
Synthetic Datasets				
Friedman	1.646	**1.479**	1.6551	1.6382
FriedmanDrift	1.699	**1.533**	1.7848	1.6914
Planes2D	**0.855**	0.866	0.9799	0.8837
Real-World Datasets				
Abalone	**743635**	791118	832390	789657
Superconductivity	29066.4084	**27726.3299**	27726.5335	27726.6728
House8L	**2924887.6685**	3167848.9818	3168696.9625	3168264.3407
BikeSharing	72.4493	74.3894	74.4615	**63.2711**
Average Rank	2.14	**1.86**	3.71	2.29

Table 4: Root Mean Squared Error (RMSE) Results. Lower is better. Bold indicates the best performance per dataset.

Data stream	ARFStandard	DWARF	ARFER	DWARFER
Synthetic Datasets				
Friedman	2.0915	**1.8734**	2.1125	2.0892
FriedmanDrift	2.0811	**1.9855**	2.3247	2.0520
Planes2D	**1.1051**	1.1143	1.2373	1.1280
Real-World Datasets				
Abalone	**1.9985**	2.0132	2.0414	2.0028
Superconductivity	4236910.7897	**4041361.0527**	4041361.1500	4041361.2500
House8L	**438483864.7492**	475024208.7998	475024208.8705	475024208.8101
BikeSharing	108.2697	108.5030	196.5971	**94.2829**
Average Rank	2.14	**1.86**	3.71	2.29

Table 5: R-squared (R^2) Results. Higher is better. Bold indicates the best performance per dataset.

Data stream	ARFStandard	DWARF	ARFER	DWARFER
Synthetic Datasets				
Friedman	0.820	**0.853**	0.8188	0.8228
FriedmanDrift	0.803	**0.837**	0.7860	0.8077
Planes2D	**0.940**	0.939	0.9211	0.9360
Real-World Datasets				
Abalone	0.6084	0.6100	0.5990	**0.6140**
Superconductivity	-15299854033.8258	-13920151163.6183	-13920151163.6134	**-13920151163.6035**
House8L	**-68856277.7579**	-80810500.3012	-80810500.3253	-80810500.3047
BikeSharing	0.6422	0.6437	-0.1748	**0.7298**
Average Rank	2.57	**1.86**	3.71	**1.86**

Results show that the proposed `DWARF` model offers competitive performance and notable advantages over the standard ARF-Reg, particularly in scenarios involving concept drift. However, the benefits of its dynamic weighting mechanism vary depending on the characteristics of the data stream. Figure 1 illustrates the progressive Mean Absolute Error (MAE) on the stable *friedman* stream. In this case, `DWARF` achieved a final MAE of 1.479, outperforming the standard ARF's 1.646. The lower and smoother MAE trajectory for `DWARF` suggests a more stable and effective learning process even without significant drift. The additional experiments provide a more nuanced picture of `DWARF`'s behavior. On the *friedman_drift* dataset (Figure 2), the advantage of the dynamic approach is more pronounced. `DWARF` maintains a consistently lower MAE throughout the stream and finishes with a significantly better score (1.533 vs. 1.699), demonstrating its effectiveness in adapting to changing data distributions. Conversely, on the *planes2d* dataset (Figure 3), the standard ARF performed marginally better, achieving a final MAE of 0.855 compared to 0.866 for `DWARF`. This suggests that on stable streams where a baseline model is already highly effective, the added complexity of the dynamic weighting mechanism may not provide a benefit and can introduce a minor performance overhead.

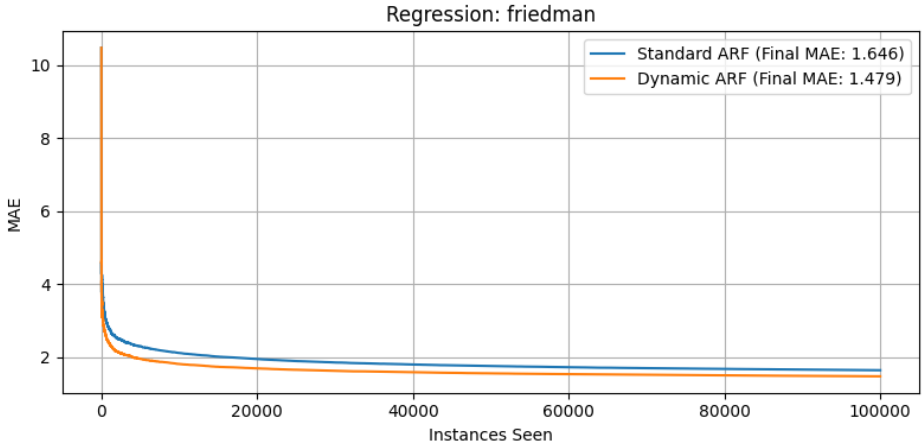

Figure 1: Standard ARF vs. DWARF Regressor with Dynamic Weights evaluated on friedman

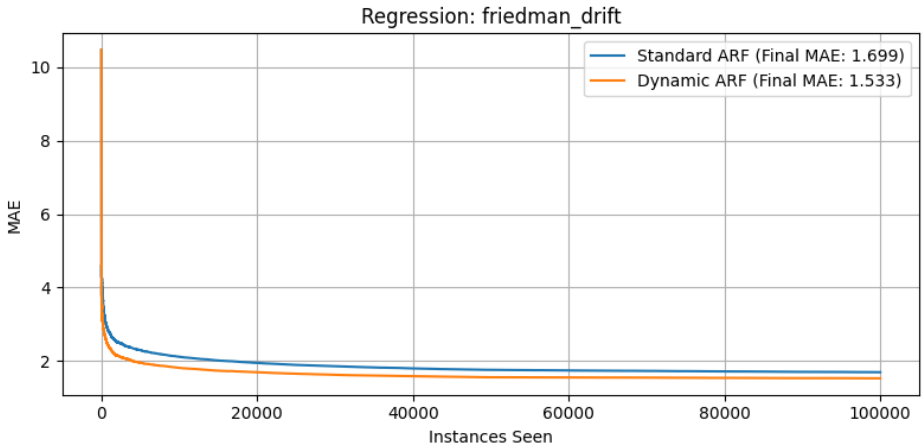

Figure 2: Standard ARF vs. DWARF Regressor with Dynamic Weights evaluated on friedman drift

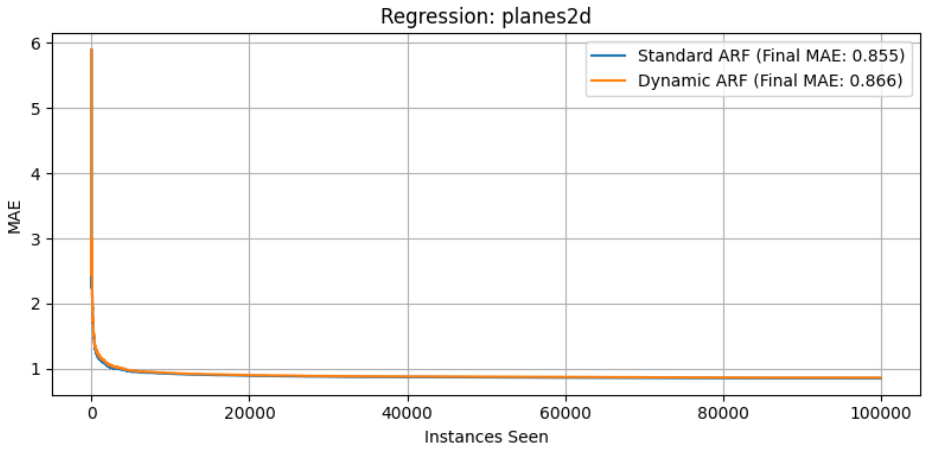

Figure 3: Standard ARF vs. DWARF Regressor with Dynamic Weights evaluated on planes2d

On the Abalone dataset, there is little difference between the performance of each method, with Standard ARF having the lowest MAE score (0.6084) by a slight margin, and DWARFER achieving the second-best result (0.6140). The ARFER and DWARFER logic, including dynamic ensemble resizing, could be well tested for this stream, as we can clearly observe the instability of the number of models used throughout the processing time. For ARFER, the ensemble at first shrinks from the initial 10 to 7 base learners before increasing to 14 and after another 1000 instances reaching 20 base learners (which is the maximum limit in this setting), with slight variations until the last processed instance (see Figure 5).

Interestingly, the changes in the number of base learners used in DWARFER differ. As shown in Figure 6, DWARFER's ensemble size never drops below the initial 10 learners. This key difference highlights the stabilizing effect of its dynamic weighting mechanism. ARFER's initial shrinkage is a direct result of its "accuracy-drop" pruning rule; without the stability of weighted predictions, several of its initial learners perform poorly during the stream's early instability, triggering warnings and subsequent performance monitoring. Their failure to recover leads to them being pruned for sustained performance degradation. In contrast, DWARFER's dynamic weighting acts as a protective shield. By improving the overall predictive accuracy of the ensemble, it prevents its individual trees from experiencing the severe performance drops needed to trigger the same pruning rule, giving them the resilience to adapt rather than be removed. After this initial phase, both models grow to their maximum size, and the later fluctuations are driven by a different rule: pruning the worst-performing model to make space for a new one when a drift is detected at maximum capacity.

The evaluation on the real-world BikeSharing dataset, which contains significant concept drifts, highlights the distinct behaviors of the proposed algorithms. Figure 7 shows the progressive Mean Absolute Error (MAE) for all four models. While all models initially struggle with a drift occurring around instance 2,500, `DWARFER` demonstrates superior adaptation, achieving a substantially lower and more stable MAE post-drift compared to `ARFER` and the other baselines.

The reason for this performance difference is revealed by analyzing the ensemble size evolution, shown for `ARFER` in Figure 8 and for `DWARFER` in Figure 9. Both algorithms react to the initial drift

by rapidly increasing their number of base learners toward the maximum of 20. However, their subsequent behavior diverges significantly. DWARFER (Figure 9) exhibits an explosive growth phase and then maintains a stable ensemble of 20 learners for the remainder of the stream. In contrast, ARFER (Figure 8) enters a state of continuous churn, where its ensemble size constantly fluctuates at or near the maximum limit.

The source of this divergence is the powerful **indirect influence** of DWARFER's dynamic weighting mechanism. It is crucial to note that the weighting does not directly control the decision to add or remove learners; this is governed by the drift detectors. Instead, the weighting alters the stability of the learning environment. Figure 10 provides a granular view of ARFER's instability, revealing a near-constant stream of drift warnings (orange) and confirmations (gray) after the initial drift. Because ARFER relies on a simple average prediction, its overall performance is more susceptible to underperforming trees, leading to a volatile error signal. This volatility causes individual learners to frequently trigger drift alerts, forcing the algorithm into a continuous cycle of pruning its worst-performing tree to make space for a new background learner.

Conversely, DWARFER's dynamic weighting creates a more stable system. By effectively amplifying the contribution of well-adapted learners and suppressing the impact of poorly-adapted ones, it improves the overall predictive accuracy of the ensemble. This enhanced stability reduces the error variance experienced by the individual trees, making it far less likely for their drift detectors to trigger after the initial adaptation is complete.

Notably, on the Superconductivity and House8L datasets, the conditions required to trigger ensemble resizing—such as confirmed concept drifts or significant accuracy degradation—were not met, and as a result, the number of base learners for ARFER and DWARFER held constant at the initial count of 10 for the duration of the evaluation.

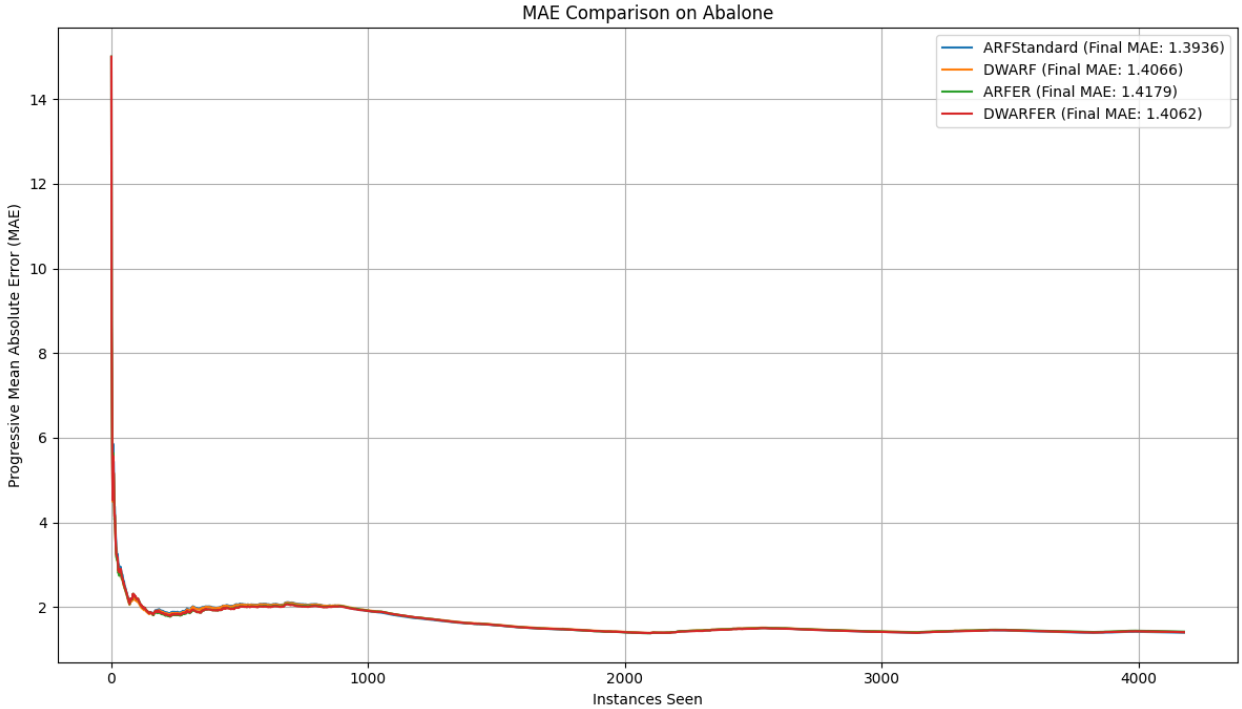

Figure 4: MAE for ARF algorithm variants (Abalone)

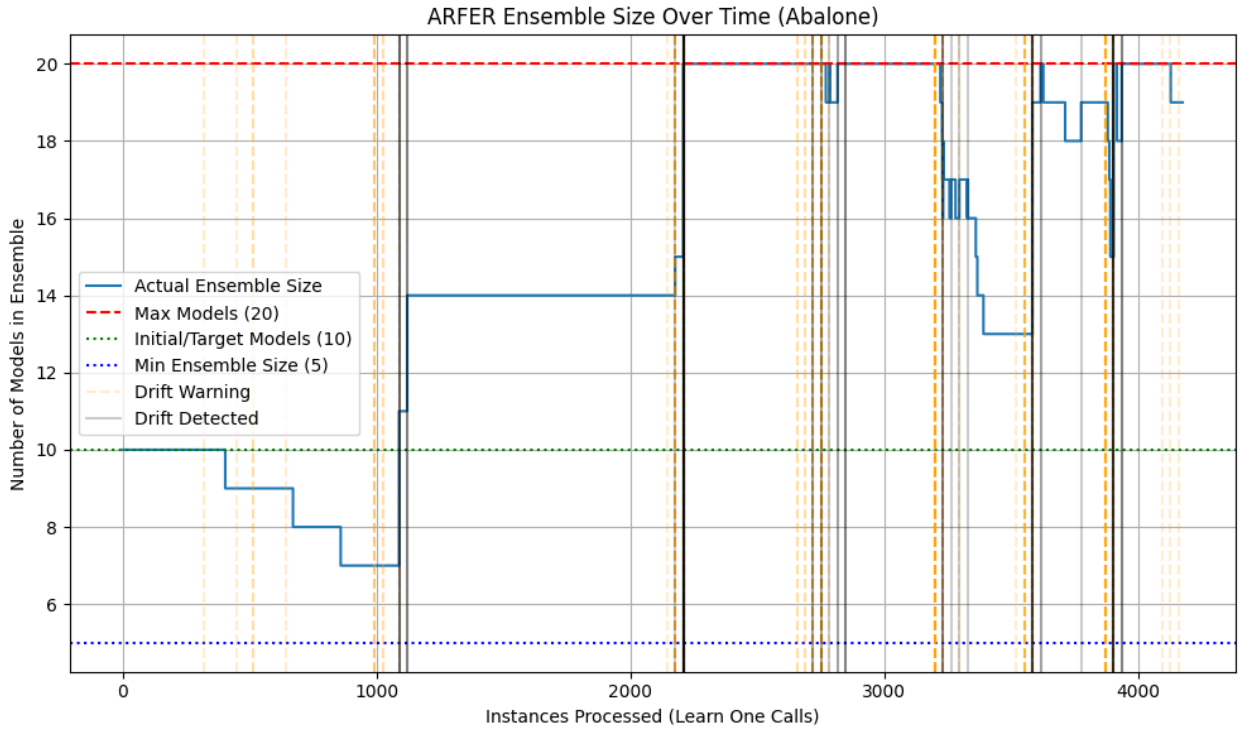

Figure 5: ARFER Ensemble Size with Drift Warnings and Confirmations (Abalone)

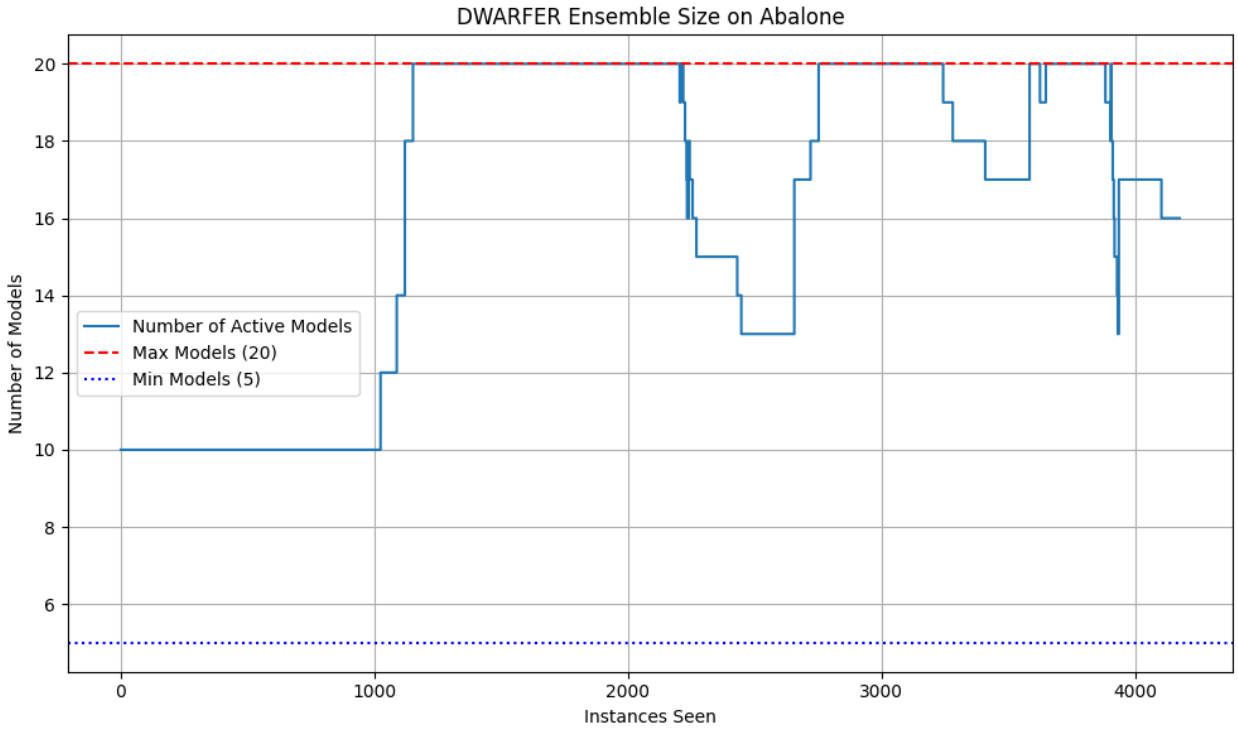

Figure 6: DWARFER Ensemble Size (Abalone)

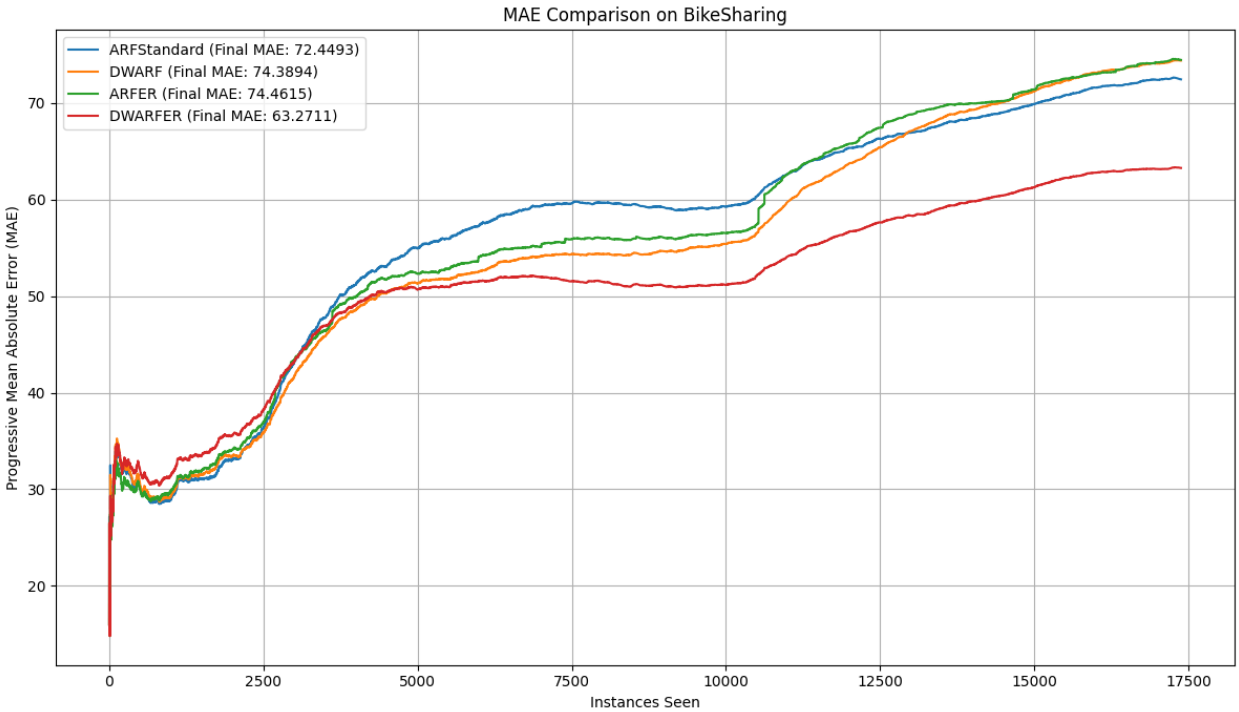

Figure 7: MAE for ARF algorithm variants (BikeSharing)

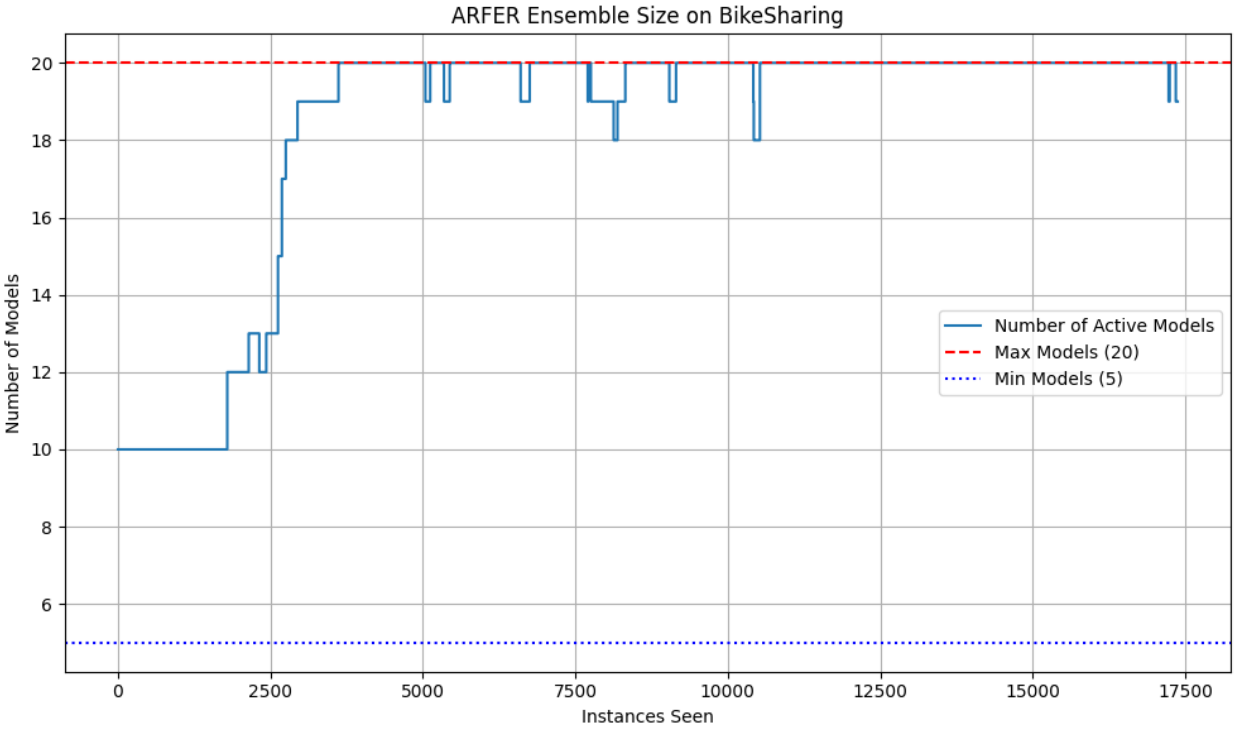

Figure 8: ARFER Ensemble Size (BikeSharing)

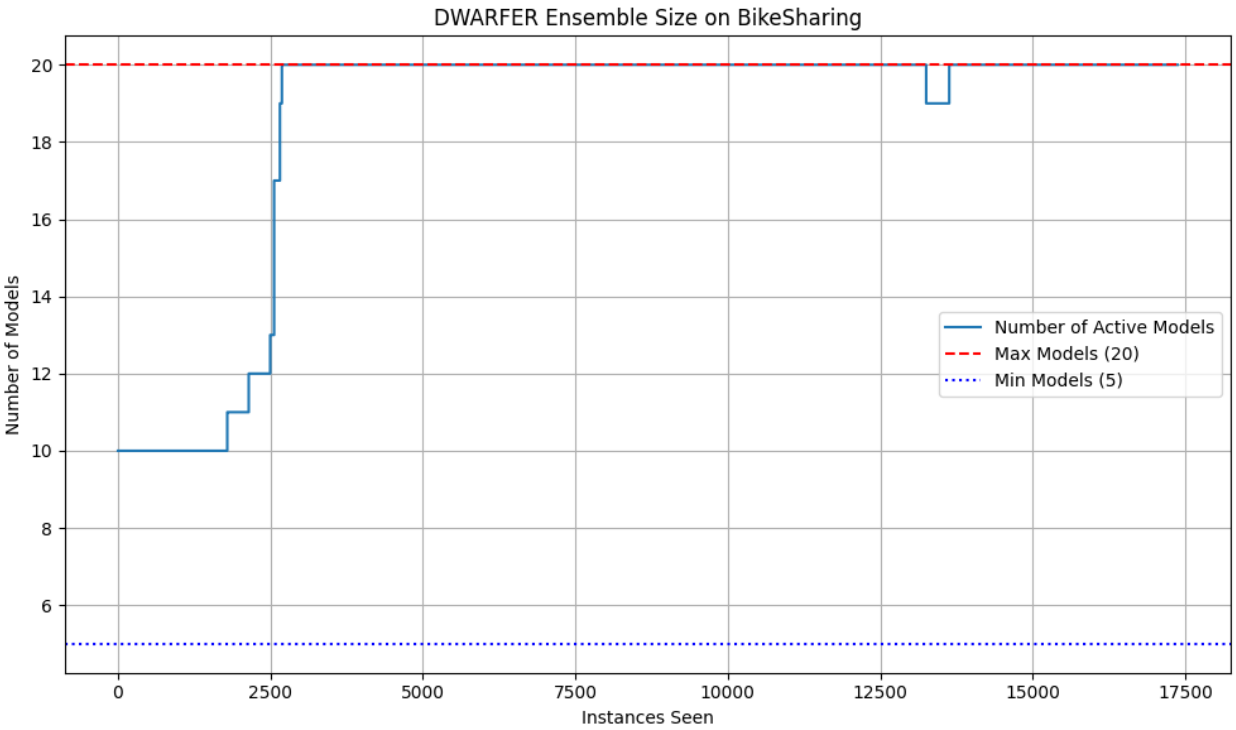

Figure 9: DWARFER Ensemble Size (BikeSharing)

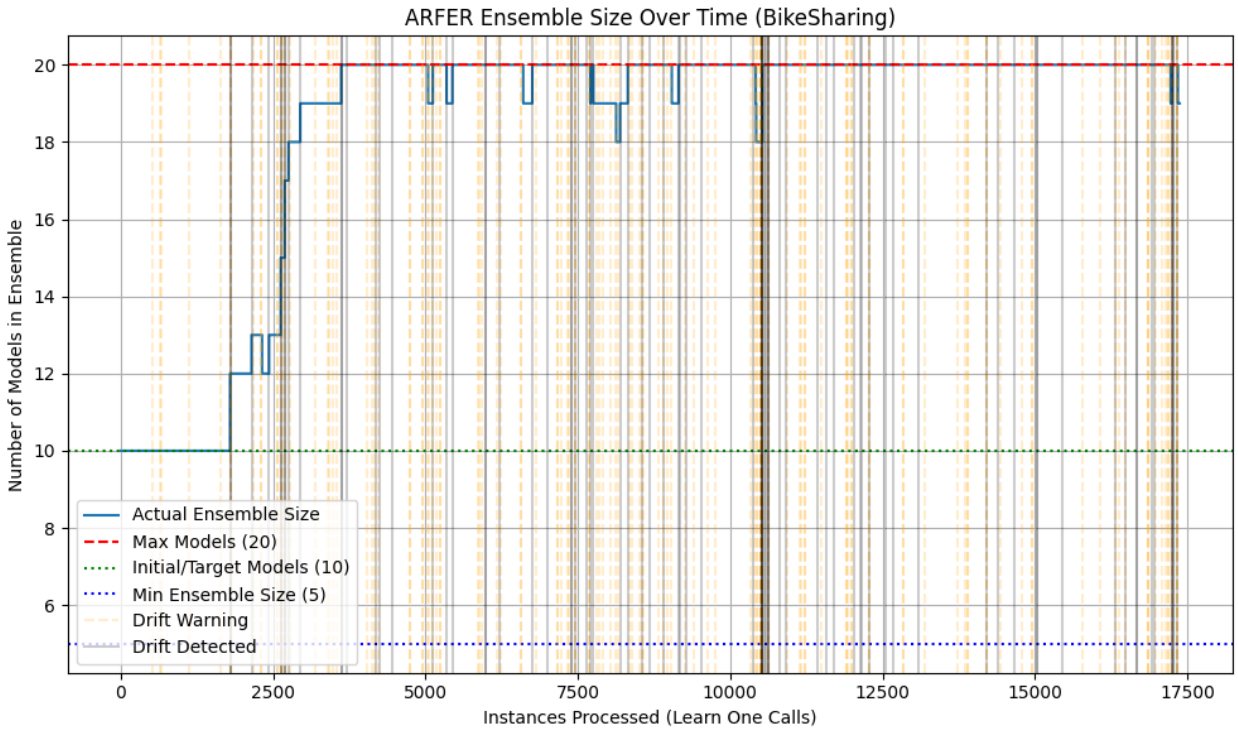

Figure 10: ARFER Ensemble Size with Drift Warnings and Confirmations (BikeSharing)

4 Conclusions

In this project, we proposed and evaluated several extensions to the Adaptive Random Forest (ARF-reg) framework for data streams for regression tasks. Motivated by the limitations in the original ARF-Reg algorithms such as fixed ensemble size and equal contribution of base learners' predictions, we introduced three custom extensions including dynamic ensemble resizing (ARFER), dynamic tree weighting based on recent prediction performance (DWARF), and a unified model combining both mechanisms (DWARFER).

Empirical results demonstrate that our enhanced ARF-Reg version often achieved consistently lower progressive Mean Absolute Error (MAE) than the baseline standard ARF-Reg, while simultaneously reducing ensemble complexity when appropriate. In particular, we observed that the sole mechanism of weighting votes of the base learners introduced in DWARF, can significantly improve the ensemble performance (average rank of 1.86) over standard ARF-Reg in many cases. It is also significantly better than the mechanism of ensemble resizing proposed in ARFER, which performed on average the worst out of all 4 algorithms considered in the study. This is a consequence of some fixed conditions for increasing or reducing the number of base learners, directly connected with the drift detection. The combination of the two mechanisms, DWARFER, was only better for one dataset (BikeSharing), but it performed similarly if not worse than the Standard ARF-Reg in most cases. Additionally, dynamic ensemble resizing in ARFER showed promising behavior in tracking concept drift by allocating model capacity dynamically. However, its pruning strategy may have been too aggressive or insufficiently adaptive in some cases, which could explain its lower overall performance. Interestingly, while DWARFER aimed to leverage the strengths of both adaptive weighting and ensemble resizing, its performance suggests that combining both mechanisms

naively does not guarantee additive improvements. The interaction between model weighting and resizing of the ensemble may require more careful coordination, especially under rapidly evolving data distributions.

Further research should focus on improving the dynamic weighting mechanism, especially on finding the most effective function for computing the vote weights. This function could take into account not only recent prediction accuracy but also additional factors such as how recently a tree was trained (e.g., after a drift), the stability of its predictions, or its confidence. Exploring such enhancements may lead to better adaptation to changing data distributions and more robust ensemble performance.

References

[1] P. Domingos and G. Hulten, "Mining high-speed data streams," in *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '00, Boston, Massachusetts, USA: Association for Computing Machinery, 2000, pp. 71–80, ISBN: 1581132336. DOI: 10.1145/347090.347107. [Online]. Available: https://doi.org/10.1145/347090.347107.

[2] E. Ikonomovska, J. Gama, and S. Džeroski, "Learning model trees from evolving data streams," *Data Mining and Knowledge Discovery*, vol. 23, pp. 128–168, Jul. 2011. DOI: 10.1007/s10618-010-0201-y.

[3] H. M. Gomes, A. Bifet, J. Read, *et al.*, "Adaptive random forests for evolving data stream classification," *Machine Learning*, vol. 106, pp. 1–27, Oct. 2017. DOI: 10.1007/s10994-017-5642-8.

[4] H. M. Gomes, J. P. Barddal, L. Boiko Ferreira, and A. Bifet, "Adaptive random forests for data stream regression," Apr. 2018.

[5] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," vol. 7, Apr. 2007. DOI: 10.1137/1.9781611972771.42.

[6] A. Cano and B. Krawczyk, "Rose: Robust online self-adjusting ensemble for continual learning on imbalanced drifting data streams," *Machine Learning*, vol. 111, no. 9, pp. 2561–2599, 2022. DOI: 10.1007/s10994-022-06168-x. [Online]. Available: https://doi.org/10.1007/s10994-022-06168-x.

[7] Y. Sun, H. M. Gomes, B. Pfahringer, and A. Bifet, *Evaluation for regression analyses on evolving data streams*, 2025. arXiv: 2502.07213 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2502.07213.

[8] W. Nash, T. Sellers, S. Talbot, A. Cawthorn, and W. Ford, *Abalone dataset*, 1995. DOI: 10.24432/C55C7W. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/abalone.

[9] H. Fanaee-T and J. Gama, "Event labeling combining ensemble detectors and background knowledge," *Progress in Artificial Intelligence*, vol. 2, no. 2, pp. 113–127, 2014. DOI: 10.1007/s13748-013-0040-3.

[10] L. Torgo, *House8l dataset*, Derived from the U.S. Census, 1990. Available at: `https://www.dcc.fc.up.pt/~ltorgo/Regression/census.html`, 1990.

[11] K. Hamidieh, "A data-driven statistical model for predicting the critical temperature of a superconductor," *Computational Materials Science*, vol. 154, pp. 346–354, 2018. DOI: `10.1016/j.commatsci.2018.07.052`.

A Source Code

The GitHub repository with all the source code and datasets is available at `https://github.com/Bartosz7/MLDS-2025L-SmartARFs`.

B Task Distribution

Task	Task Description	Assigned Member(s)
T1	Implemented baseline Adaptive Random Forest (ARF) regressor using `HoeffdingTreeRegressor` as base learner.	Filip Kucia, Bartosz Grabek
T2	Developed Smart ARF with dynamic ensemble resizing, incorporating pruning mechanisms based on performance metrics.	Filip Kucia
T3	Integrated dynamic weighting mechanism into ARF, adjusting tree contributions based on recent performance.	Bartosz Grabek
T4	Combined dynamic weighting and ensemble resizing into a unified Smart ARF model.	Filip Kucia
T5	Conducted experiments and evaluations on various ARF models to assess performance under different drift scenarios.	Bartosz Grabek, Filip Kucia
T6	Documented findings and prepared final report summarizing methodologies and results.	Bartosz Grabek, Filip Kucia

Table 6: Distribution of Project Tasks Among Team Members

C Updates to Preliminary Report

As far as the obligatory requirements are concerned:

- we extended the analysis to include evaluation on Real Data Streams
- we included plots for detecting drift
- the data, the results and the report are now included in the GitHub repository

For the updates to the report w.r.t. other comments, pelase see the table below.

#	**Initial Feedback**	**Updates**
1	Report was submitted after the deadline.	Acknowledged. Submission was slightly delayed but all required elements are now completed.
2	Use *ARF* for classification and *ARF-Reg* for regression. Also revise the claim about equal voting.	Terminology was revised throughout the paper. It is now clearly stated that ARF (classification) uses accuracy-based weighting, whereas ARF-Reg averages predictions. This distinction is made explicit early in the report.
3	Clarify whether ARF-Reg was reimplemented or extended.	Clarified in Section 2 that we implemented and used an extension of the original ARF-Reg as baseline for our experiments.
4	Minor typo: "existingriver".	This and other typos corrected. All sections were spell-checked.
5	Algorithm 1 is not commented explicitly, and the link to `HoeffdingTreeRegressor` unclear.	A new paragraph was added to clarify that RFTree is an alias for `HoeffdingTreeRegressor` and to describe its role in the ensemble.
6	Section 2.1 title and phrasing should clearly distinguish existing method vs. our extension.	Section 2.1 title was changed. The phrasing now clearly states that we implemented an extension of ARF-Reg and used it as baseline.‘
7	"If replacement is impossible... tree is reset" – why would this happen if background trees always exist?	That is correct. This case is non-existent and we removed this consideration altogether.
8	For each figure, show the data stream and ensemble size.	Captions for all plots were revised to specify the dataset and number of trees used in the ensemble.
9	Extra points possible for comparing weighting vs. ensemble resizing.	A detailed comparison was added to the evaluation section. The DWARF variant outperformed others on most datasets, supporting the importance of dynamic weighting.